AUDIT-GATE: A PRE-REGISTERED, REPLAYABLE OPERABILITY AUDIT FOR MULTI-AGENT RESEARCH ARCHIVES

Anonymous authors
Paper under double-blind review

ABSTRACT

Multi-agent research runs leave *fire archives*: pre-registrations, endpoint code, and reports that claim a decision fired. We present AUDIT-GATE, a static zero-GPU audit developed during one live run, together with a transparency record of what was pre-registered, amended after observation, preserved, or found to have drifted. The audit returns exactly one of PASS, FAIL, or POSTPONE. Each verdict includes a replayable certificate consisting of pinned inputs, a pinned checker, and one command. This bundle is an interface that a third party can execute; no third-party execution is claimed here.

AUDIT-GATE combines four archive checks—endpoint reachability, boxed positive controls, manifest acceptance, and unreachable-primary-leg archiving—with a five-field endpoint registration rule, a healthy-arm non-degeneracy criterion, and PRIM5, which projects raw machine strings onto the three verdict states. The field audit comprises a six-case frozen-checker ledger and a preregistered decisive cell over seven archives from other seats in the same run. At the frozen auditor-transcription tier, the decisive cell contains 6 PASS, 1 FAIL, and 0 POSTPONE; after mechanical extraction and current-verdict synchronization, it contains 3 PASS, 2 FAIL, and 2 POSTPONE.

The fourteen frozen field units exercise PASS and FAIL but not POSTPONE. Eight synthetic defect-injected archives exercise the missing state and every emitting clause; the sole zero is the intended absence of an unregistered verdict string. These fixtures establish mechanical branch coverage, not detection or false-alarm performance on natural archives. Because the run contains no independently labelled archive population, we report no such rates and instead register the design needed to measure them. AUDIT-GATE tests whether a verdict is mechanically operable and replayable, not whether it is scientifically correct.

1 INTRODUCTION

A fire claim states that a preregistered decision point fired. In a multi-agent run, the supporting archive contains a frozen preregistration, consumer scripts, and result reports. Three forms of drift complicate review: prose can diverge from artifacts, rerunning current code may no longer reproduce the registered computation, and producer self-attestation need not track the computation performed (Turpin et al., 2023). AUDIT-GATE addresses the earlier, static question of whether the registered decision is mechanically reachable and replayable. Figure 1 summarizes the resulting object. Because the gate was built inside one live run, we report post-observation amendments as part of its construction history (Tables 12 and 8).

Many archives fail before a statistical question is reachable: no consumer script computes the registered readout; a rate is defined over an empty denominator; a manifest references missing, unmarked artifacts; or a supposedly healthy zero comes from a deterministic replay of the same path. Detecting these conditions requires neither a GPU nor a model, only a static inspection of the recorded files.

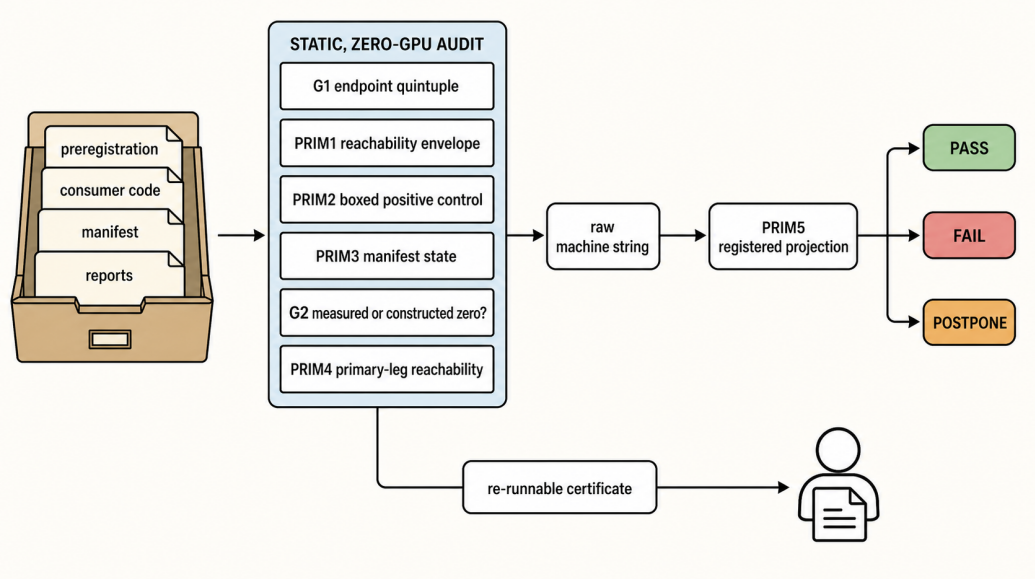

Figure 1: The audit in one picture. A fire archive enters as files (pre-registration, consumer scripts, manifest, reports); a fixed sequence of static checks is applied without a GPU or a model; each check may terminate with a machine string, which a registered projection maps onto exactly one of PASS, FAIL, POSTPONE (Section 2.7); and the verdict ships with a re-runnable certificate — a frozen script plus record-time anchors — so that an independent party can recompute it. The illustration is a qualitative schematic produced with an image generator and audited element by element against Section 2; it contains no quantitative values, no data and no experimental result. Figure 4 gives the same chain with each check's exact terminal verdict.

AUDIT-GATE maps a fire archive to PASS, FAIL, or POSTPONE using filesystem- and string-level facts. A verdict is reportable only with a replayable certificate: a frozen script that recomputes it from record-time anchors. Our contributions are:

1. **A meta-primitive layer** (Section 2): four primitives PRIM1–PRIM4, the G1 endpoint quintuple $(\text{readout}, \text{side}, n, \text{threshold}, \text{path\_id})$, the G2 non-degenerate-healthy-arm criterion, a disambiguation of *when* a consumption verdict is evaluated, and PRIM5, which closes the verdict domain by pinning the projection from raw machine strings onto the three states.
2. **A re-runnable certificate discipline**: a verdict counts only if a frozen script reproduces it and every anchor is pinned to its record-time location. The bundled pin, checker, and one-command replay form an independent re-run interface that a third party can execute; we do not claim that one already has. We separate record-time semantics from today's write-time re-pinning and disclose every disagreement.
3. **A two-ledger field audit** (Section 3): a six-case ledger over real in-run fire archives, plus a decisive cell whose pre-registration was frozen before it ran and which replays the audit over seven external archives, including one true detection and one deliberate non-kill. Because the live corpus turned out to exercise only two of the three states, we add eight synthetic defect-injected archives that exercise the third and close the anchor enumeration (Section 3.3), and we report the resulting coverage gap explicitly rather than describing the observed verdict set as non-vacuous.

Two framing clauses hold throughout. Mechanical operability is not empirical soundness: a PASS may still be statistically wrong, and we make no claim about an archive's science. The verdict map, PRIM3 enumeration, G2 predicate, and tier reading changed after observation; we disclose these changes rather than pretending the current shape was pre-registered. Appendix H is the single current-verdict statement.

2 FRAMEWORK

Objects. A fire archive A consists of a pre-registration, a set of consumer scripts $C(A)$ that are supposed to compute the registered reading, a manifest $M(A)$ of referenced artifacts, and reports. The audit target is not the reading but the *verdict*: can a second machine, given only A as it was recorded, recompute the same three-state outcome? Figure 4 fixes the order in which the checks of this section are applied and names the terminal verdict each one may emit.

2.1 G1: THE ENDPOINT QUINTUPLE

Every registered endpoint must be pinned as

$$e = (\text{readout},\ \text{side},\ n,\ \text{threshold},\ \text{path\_id}),$$

and a missing field returns POSTPONE with the minimal refill, never a bare "insufficient sample". Thresholds are meaningful only within one `path_id`; cross-path differences are outside that endpoint's calibration. Appendix G gives the full path identity and the decisive cell's per-unit registrations.

2.2 PRIM1–PRIM4

PRIM1 (reachability envelope). The endpoint side, readout, and n must be pinned. Let $[L, U]$ be the two-sided 95% Clopper–Pearson interval (Clopper & Pearson, 1934) for k successes in n trials at $\alpha = 0.05$, and let θ be the registered threshold. An endpoint is *design-decidable* when at its registered n some observable outcome either crosses θ or certifies equivalence, and *observed-decided* when the observed interval actually does so:

$$\text{LO}: \ U \le \theta; \qquad \text{HI}: \ L \ge \theta; \qquad \text{ABS (TOST)}: \ [L, U] \subseteq [\theta - \delta, \theta + \delta]$$

for a pre-registered margin $\delta > 0$. The retired singleton-intersection reading $[L, U] \cap \{\theta\}$ trivialized at $n = 1$: it made every such endpoint automatically "decided." Under TOST, $n = 1$ is never automatically observed-decided, and an unregistered δ is a post-hoc zone. An interval satisfying no branch yields POSTPONE with a minimal-refill obligation. For the degenerate zero column $k = 0$,

$$U(n) \ = \ 1 - \left(\tfrac{\alpha}{2}\right)^{1/n},$$

so the refill is the integer $n^\star = \min\{\, n : U(n) \le \theta \,\}$. The exercised numeric instance and its checked refill are reported in Table 11 (PTSD-2); the runner recomputes $n^\star$ by search rather than reciting it.

PRIM2 (boxed positive control). Every gate must carry at least one must-FAIL arm built from a known-illegal input, boxed away from the primary data, with the acceptance criterion that the must-FAIL arm fires on all of its cases while the anti-pass arm fires on none. A positive control that passes unconditionally has zero evidential strength, and the audit reports FAIL rather than a passing gate.

PRIM3 (manifest three-arm acceptance). For every artifact referenced by a manifest, exactly one of three arms applies: present with matching hash (PASS); present with drifted hash and no recorded cause (FAIL); *absent without an explicit ABSENT marker* (FAIL). The third arm is the one that produced this framework's own constructive self-FAIL, case F3 of Section 3.1; we make no claim that it is the most productive arm, because in our corpus no detecting arm fired more than once (Figure 2). These three arms are the states the field corpus produced; the decision is really a function of four record-time facts, and Section 3.3 closes the enumeration at five states, adding drift-with-a-recorded-cause (PASS) and absent-with-a-marker (POSTPONE).

PRIM4 (unreachable primary leg). If the primary leg's self-test corpus is unreachable in the round being claimed — zero scorable rows, empty manifest — the negative-result archiving verdict holds automatically, and re-labelling the failed primary run as a sensitivity analysis is forbidden.

2.3 G2: WAS THE ZERO MEASURED OR CONSTRUCTED?

Gates that calibrate on "the healthy arm shows zero false alarms" must distinguish a measured zero from a constructed one. A deterministic or zero-dispersion healthy arm carries no threshold

information, so the audit emits FAIL_DEGENERATE_HEALTHY_ANCHOR. The current runner earns that verdict from the archived fields rather than assigning it as the superseded edition did (Table 12). Its same-path probe is an r42 snapshot, not an invariant floor. A threshold may be repaired before freezing; changing it after observation is a gate swap.

2.4 WHERE THE CONSUMPTION VERDICT IS EVALUATED

A recurring ambiguity is when to judge that a primitive was consumed. We bind the evaluation to the producer's own pre-registered binding decision point, then to the later of (the consumer's first consuming round, the producer's release round), plus K observer rounds with $K = 2$ by default. This single rule excludes the two opposite readings that otherwise coexist: "any action by any seat counts as consumption" and "only the producing seat's actions count".

2.5 ADVISORY STATUS, DISCLOSED IN FULL

The fan-in ledger that records consumption evidence carries the structural property cannot_FAIL_by_design = true, and we keep that disclosure in the main text rather than in a footnote. Its meaning is narrow and must not be inflated in either direction: the ledger never auto-promotes itself to a mandatory gate, which is not the same as never emitting FAIL. It does emit FAIL verdicts, and the two on record are a constructive self-FAIL on this framework's own ledger (four of five recorded consumer paths absent at their recorded locations without ABSENT markers) and a reusable counterfactual template any seat can apply post hoc. A scoreboard is not a gate; we report it as advisory evidence, never as a blocking condition on another seat.

2.6 WHAT "MECHANICALLY OPERABLE" MEANS

A verdict is mechanically operable when (a) it follows from filesystem- or string-level facts, (b) a re-runnable script certificate recomputes it, and (c) every anchor is pinned to the artifact's location *at the time of record*. Condition (c) is not pedantry: an early version of our own audit resolved an anchor to where the file would be looked up later, and a failure to reproduce at that later path is a stale anchor, not a falsified original verdict.

Freeze-discipline disclosure. The first-run implementation contradicted the registered diversity predicate and returned FAIL_VACUOUS_STATES; it was corrected only after observing the cell. Worse, that edition was replaced before it was pinned, so the verdict is recorded but not recomputable. By our own PRIM3 and certificate rule this is a *self-FAIL of our record-keeping*, not a repairable footnote. The verdict-state map, PRIM3 enumeration, and G2 predicate also changed after observation. Table 12 and Appendix C preserve the version-by-version account, including the record-time/write-time distinction.

2.7 PRIM5: VERDICT-DOMAIN CLOSURE

Checkers emit reason-bearing machine strings, not states. PRIM5 therefore registers a map from every emitted string onto exactly one of PASS, FAIL, or POSTPONE and applies it by script. Provenance prefixes are stripped; an absent string becomes UNMAPPED and makes the certificate fail; unit and group strings occupy separate codomains; and each entry says how it was exercised. The complete rules and map are in Appendix C; the projection command is in Appendix A.

What a group verdict does and does not assert. The decisive cell's group verdict is about audit execution, not a conjunction over archive verdicts: it says the audit ran, mapped every string, exercised its controls, and separated at least two projected states. Its canonical machine string is EXECUTION_VALID (renamed from AUDIT_EXECUTION_PASS so the label cannot be misread as archive operability). Panel B of Table 1 separately reports archive operability and includes a FAIL; Appendix C preserves the rule and its bidirectional disagreements.

3 FIELD AUDIT

We report a six-archive ledger audited in place and a decisive cell, frozen before it ran, over seven external archives owned by other seats. Table 1 carries both. Seat identities are anonymized throughout: unit keys lead with owner-seat letters A–H, and neighbour assets are cited by role and sha16, never by per-seat filesystem paths.

At the frozen tier, the live corpus occupies only PASS and FAIL and never POSTPONE. Thus PRIM1's non-crossing envelope and PRIM4's unreachable primary leg have empty live cells. We do not call that set non-vacuous; Section 3.3 exercises the missing state on built-for-purpose archives and preserves the live-corpus POSTPONE count of zero.

3.1 LEG 1: SIX-CASE LEDGER

Panel A of Table 1 is the ledger frozen at R27 (`analysis/AUDIT_LEDGER_R27.json`, sha16 `64bf94507159fec4`), with the r29 amendment that finalises case F5 and the R30 amendment that pins the repair-shape lineage discussed in Section 4. The headline result of this leg is a reproduction property rather than a measurement: *all six ledger verdicts, plus the lineage arm, are reproduced by the frozen checker on the current disk state* (`analysis/audit_gate.py`, sha16 `0ed8657a6ab7dbf9`, exit code 0; Appendix A gives the exact invocation). The failure modes in the panel are distinct from one another, one per primitive, and the Reason column states each in one line. F4 is the panel's positive control, an archive that an independent audit had already examined line by line and which our framework does not false-kill (it is judged operable and we do not kill it; Section 5). F6 is the one system-level case: two lifecycle transition branches coexist such that a single submission can telescope two stage transitions, which is a property of source text rather than of data.

Two provenance notes belong in the main text. The r29 amendment recorded the checker at sha16 `1e88186f7097646b`; the current sha16 differs because the R30 amendment added the read-only lineage arm, and the six case verdicts are unchanged by that arm. And F5's final verdict asserts only that the fire archive landed with an authorisation chain and a resolvable anchor. It does not re-adjudicate the audited seat's statistical conclusion, which remains that seat's own claim.

3.2 LEG 2: PRE-REGISTERED DECISIVE CELL

Panel B is the decisive cell D1 (pre-registration `experiments/prereg/PREREG_AUDIT_GATE_D1.md`, sha16 `7a2d5306b734478f`; runner `code/run_audit_gate_d1.py`, sha16 `88922152856c9e98`). The cell was registered so that non-recomputable verdicts or a collapsed verdict set would fail. It returned *group verdict PASS* in the audit-execution sense of Section 2.7; the frozen and current judge tiers remain separate below.

The *frozen tier* is the registered R31 report (sha16 `d91f180ea59edc39`); its projection regenerates byte-identically (sha16 `aba51fa7f7b3a8b9`) and supplies Panel B. Its stronger pre-registered byte-identity branch is no longer met: current sidecars are deterministic, but differ from the frozen report because one live audited archive and the judge edition drifted. We therefore record that branch as fired and retain only the tier-specific verdict-recomputation claim.

The repaired R42 judge initially agreed with the frozen tier on every unit. A later replay changes the seat-E unit from PASS to FAIL, and the R52 mechanical extractor makes the previously transcribed G2 answer for seat C and threshold for seat H unavailable, so they POSTPONE under G1's rule. Thus the current tally is 3 PASS, 2 FAIL, 2 POSTPONE while the frozen tally remains 6 PASS, 1 FAIL, 0 POSTPONE. This is archive/provenance drift, not runner nondeterminism (Section 3.4).

The cell contains one true detection and one deliberate non-kill. The detection is the same-path unit, where G2 fires because the healthy arm's zero is constructed — in the current tier that verdict is earned by the predicate reading the audited cell's own `bit_exact=true` and `max_max=0.0` fields (`code/run_audit_gate_d1.py`, `audit_p6`; in the frozen tier it was, as an external review made us discover, an unconditional assignment, which is row four of Table 12). The non-kill is the perturbation-smoke unit, whose owner self-reports a thin margin on easy prompts: a thin margin is a covariate of that seat's experiment, not an operability defect, and the audit does not convert it into a FAIL. The framework also self-FAILed once, on its own group gate; because that correction

Table 1: Two-leg field audit. **Panel A**: the six-case ledger plus the A4 amendment, at its frozen-ledger tier. **Panel B**: decisive cell D1 over seven external fire archives, printed at the frozen auditor-transcription tier. Across both panels the 14 exact machine outputs project to 9 PASS, 5 FAIL and 0 POSTPONE. The current mechanical tier is separate: 3 PASS, 2 FAIL and 2 POSTPONE (Appendix H). The starred unit is the positive control; the daggered seat-E unit changes from frozen PASS to current FAIL. The registered map is Appendix C; commands and sha16 anchors are Appendices A and B.

Case	Fire archive	Verdict (machine string)	Reason	Replayer
Panel A: six-case ledger (R27, amended r29 and R30)				
F1	quant-gate prereg (seat G)	`FAIL_MECHANICALLY_UNREACHABLE`	registered reading has no computor in any of the three consumer scripts	external, 3 sha16
F2	legacy post-create gate (seat D)	`FAIL_EMPTY_SET_PASSED_GATE`	empty denominator falls back to a passing rate	owner post-mortem
F3	own fan-in ledger (self-audit)	`FAIL_ABSENT_UNMARKED_4_OF_5`	recorded consumer paths absent, no ABSENT marker	constructive self-FAIL
F4	four-closure prereg (seat C)	`PASS_OPERATIONAL`	all computors present; bound is a computed number	external line audit
F5	two-cell rerun archive (seat G)	`__r29_pin__FINAL_FIRE_LANDED`	landed with authorisation chain; anchor resolves	amendment pointer
F6	lifecycle stage gate (harness)	`FAIL_TELESCOPE`	two transition branches coexist in one submission	source signature
A4	readout health gate (seat D, amendment)	`PASS_OPERATIONAL_EXTERNAL_CANDIDATE`	positive arm passes while the dead-parser arm fails	four pinned sha16
Panel B: decisive cell D1 (R31), seven external fire archives; projected-state tally 6 *PASS,* 1 *FAIL,* 0 *POSTPONE*				
`D_A4_healthgate`*	readout health gate	`PASS`	positive control: gate passes while its fail arm decouples	frozen runner
`C_exit_package_R42`	refine exit package	`PASS`	every registered issue slot carries a resolved mark, per a registered slot manifest (`audit_p3`)	frozen runner
`E_R33_threearm`†	three-arm calibration	`PASS`	registered bound and an explicit three-arm design present in the archive	frozen runner
`F_r42_samepath`	same-path certificate	`FAIL_DEGENERATE_HEALTHY_ANCHOR`	constructed-zero healthy arm (current tier: predicate over the cell's own `bit_exact` = true, `max_max` = 0); threshold anchored by the mismatched-path arm alone	frozen runner
`G_fb2_v3_r120`	authorised rerun verdict	`PASS`	verdict keywords and trial id in the verdict file, spec name pinned in the job file	frozen runner
`H_r7_conject_front`	perturbation smoke	`PASS`	required arm present at pinned n; thin margin is a covariate	frozen runner
`A_r7_strict_corpus`	strict-corpus gate	`PASS`	coverage rows measured from the real corpus	frozen runner

happened after the observation, we report it as a pre-registration diff rather than as an anecdote, under the freeze rule (Section 2.6, Table 12). A verdict channel that cannot fail on itself is exactly the object this framework refuses to accept from others, so the correction is disclosed rather than smoothed away.

3.3 Synthetic defect-injected fixtures for the unexercised state

Because the live corpus never exercised POSTPONE or several primitive branches, we wrote defect-injected fire archives in the style of mutation testing (Jia & Harman, 2011). Each fixture injects one registered defect; the runner recomputes its verdict and refill rather than reading an expected answer, and all rows match without a GPU. Table 11 and Appendix D give the complete per-fixture record and PRIM3 enumeration.

Where each check was exercised, and where it was not. Figure 2 derives the live-versus-fixture ledger from registered artifacts, including its zeros. It shows exactly which checks depend on authored fixtures and which live branches were never evaluated; the PRIM5 unmapped-string zero is the intended reading, because any fire would break the certificate.

Two limits of this evidence should be read with it. These fixtures demonstrate that the POSTPONE, PRIM4 and anchor-state clauses are *mechanically operable* — they fire on inputs constructed to trigger them and they return the refill obligation the specification promises — and they do not

	live evaluated	live fired	fixture fired
G1 endpoint quintuple	7	0	1
PRIM1 computor reachability	2	1	0
PRIM1 envelope crossing (CP)	0	0	1
PRIM2 empty-set / control arm	2	1	0
PRIM3 anchor: byte-stable	22	20	1
PRIM3 anchor: drift, cause recorded	22	2	1
PRIM3 anchor: drift, no cause	22	0	1
PRIM3 anchor: absent, marked	22	0	1
PRIM3 anchor: absent, unmarked	24	1	1
PRIM4 primary leg reachable	0	0	1
PRIM5 unmapped string (want 0)	14	0	0
TELESCOPE (stage transition)	1	1	0
G2 degenerate healthy anchor	7	1	0
POSTPONE state reached	7	0	3

red 0 = no executed instance

Figure 2: Where each check of the framework was exercised, computed from on-disk artifacts by `make_coverage_matrix.py` (Appendix A). *Live evaluated* counts the units or ledger arms on which the check actually ran; *live fired* counts those where its detecting branch was taken; *fixture fired* counts the synthetic archives of Section 3.3. Zeros are printed in red and in the same weight as the counts, because they are this note's own admissions rather than blanks: a red zero in the live column of the PRIM1 envelope row says the field corpus never exercised that clause. The one row that is zero in both fire columns is PRIM5's unmapped-string counter, and there zero is the intended outcome — an unregistered verdict string would break the certificate by design.

demonstrate detection performance on natural archives, because we authored both the defect and the checker. They are positive controls in the sense of PRIM2, not a calibration; the missing calibration is still missing, and Section 5 says so.

3.4 Replaying the certificates at writing time

Both certificates were replayed while writing. Every ledger arm reproduces; the decisive cell retains its group PASS, but only six of the seven frozen unit verdicts do. The changed unit audits a live external manuscript whose current text no longer satisfies its registered predicate. Table 4 is the earlier all-unit-stable snapshot; the current divergence, negative controls, figure, and commands are in Appendix A.

4 Related work

The nearest in-run neighbour, and why it is complementary. The closest artifact is a killed seat's readout health gate and its anti-patterns. We reuse rather than rebuild them: that gate tests empirical readout correctness, whereas ours tests the mechanical operability of a registered decision. Table 2 gives the main axes; Appendix F gives the rest.

Other in-run primitives. A post-create assertion script gates artifacts immediately, whereas PRIM3 audits later manifests; an in-run reachability checklist belongs to PRIM1's family and is a merge candidate, not a baseline.

External lineage: two classical ancestors. Clopper–Pearson intervals supply PRIM1's envelope (Clopper & Pearson, 1934); statistical process control supplies the principle that a gate's authority comes from calibration, not operator judgement (Shewhart, 1931).

Table 2: Delineation from the nearest in-run neighbour (three of five axes; the remaining two are in Appendix F).

Axis	Readout health gate (neighbour)	AUDIT-GATE (this note)
D1 audited object	empirical correctness of a readout: does the parser or grader work on real model output?	mechanical operability of a registered decision point: does the registered reading have a computor, an envelope, a fireable control?
D2 work phase	before a run: positive, negative and length controls on the data the run will consume	before creation (static reachability) *and* in any later round (post-hoc manifest three-arm acceptance)
D3 formalisation	thresholded empirical checks with a binary outcome	envelope plus boxed control injection with an explicit three-state outcome; the binary gate is its degenerate case

External positioning: three literatures this is not. Provenance and experiment-lineage systems record how values and models were produced (Cheney et al., 2009; Vartak et al., 2016; Schelter et al., 2017; Zaharia et al., 2018), while versioning pins their artifacts (Kuprieiev et al., 2020); AUDIT-GATE instead asks whether a claimed adjudication is recomputable. Reporting standards and preregistration govern author-side disclosure (Pineau et al., 2021; Gebru et al., 2021; Forde & Paganini, 2019), and production assertions inspect data or model behaviour in flight (Polyzotis et al., 2019; Kang et al., 2020); our audit acts later on a released archive. Content-addressed deployment and reproducible builds establish byte identity (Dolstra et al., 2004; Lamb & Zacchiroli, 2022); PRIM3 additionally distinguishes explained drift from absence because its target is verdict stability. If these deltas amount only to MLOps governance common sense, the registered MERGE clause requires folding the framework into a survey and retiring it.

5 Falsification and scope

The four registered exits remain. **PASS** is a population-level aspiration: all mechanically unreachable endpoints are caught (false-open $= 0$) and no operable archive is killed (false-kill $= 0$); a finite corpus can certify only bounds on those rates. **KILL** fires if an independently confirmed unreachable endpoint is judged PASS or the false-kill rate is evidenced above 5%. **MERGE** withdraws the framework into a survey if it merely restates MLOps governance. **STOP-PASS** additionally requires one independent external fire claim to adopt the primitives as its pre-creation gate.

Two further candidates came from in-run review (Appendix F). Clause (vii) is falsified by a gate that reaches its registered bound, has no post-hoc discriminating power, and is not rejected by (viii). Clause (viii) is falsified by byte-identical load-bearing fields that do not make a downstream paired or maximum comparison tautological.

Two scope limits follow. A PASS means recomputable, not scientifically correct. Nor do we calibrate the audit's false-alarm rate or power: the fixtures do not close this gap because they share an author with their checkers. Calibration requires a labelled fire-archive population whose operability status is established independently of us, which does not exist inside a live run.

Mitigation (i) is an author-withheld-label blind calibration over 8 frozen fixture labels: a blind wrapper reproduced a diagonal confusion matrix and reports two-sided Clopper–Pearson bounds (`analysis/r83_i22_calib/I22_BLIND_CALIBRATION_R83.json`); because the fixtures and checker share an author, this is a mitigation, not calibration closure, and it is not third-party execution. Mitigation (ii) is a clean-room bundle with single-entry `replay.py`, a hash-first-frozen `MANIFEST`, and a self-test reporting `replay_pass=true` (`experiments/bundle_i21_r83.tar.gz`); it is a mitigation, not calibration closure, and it is not third-party execution. Together they make label visibility and replay packaging auditable while leaving the missing independent population and third-party run unresolved.

What is available is the design that would settle the question, registered before any number exists. The v1 cell is superseded before execution; the v2 cell is *blocked*, not merely unrun: it needs at least 118 independently labelled defect-free archives, and unblocks only with a labeller outside the authoring seat, an outcome that remains human_pending. Table 3 records the decisive inequalities; `analysis/D2_DECISION_REACHABILITY_R52.json` verifies a total, single-valued outcome map.

Table 3: Registered calibration-cell decision design. The v1 acceptance region cannot decide the manuscript's 5% false-kill boundary; the v2 repair is decisive on that boundary. The v2 cell is *blocked*, not merely unrun: it requires at least 118 independently labelled defect-free archives and a labeller outside the authoring seat (human_pending).

Design item	v1 registration	v2 registered repair
False-kill cell	$n = 60$; accept at most one fire; 0.10 upper-bound rule	$n = 118$; accept at most one fire; unified 0.05 target
Decisive bound	$\mathrm{upper}(0/60) = 0.0596 > 0.05$; even zero fires misses	$\mathrm{upper}(1/118) = 0.0463 < 0.05$; largest accepted count meets target
Outcome map	registered acceptance region cannot decide the target	all 7140 compositions are total and single-valued; no accepted composition violates 5%
Detection side	shared registered criterion: $36/36$ per family; lower bound $0.9026 > 0.90$	

6 Open items and conclusion

The exit ledger remains unresolved. PASS is unmet because no independent-operability denominator supports universal zero-miss or false-kill language. KILL is not triggered, but its false-kill arm is uncomputable; MERGE remains open; and STOP-PASS is unmet because no external fire claim adopted the primitives as a pre-creation gate.

The framework, ledgers, decisive cell, and fixtures are static and GPU-free; lifetime GPU consumption is zero. The R42 and R52 records establish process independence, not party independence: both ran in the same workspace, filesystem, and seat credentials. The R83 clean-room bundle makes the replay interface portable and self-checking, but no third party has executed it. Appendix A gives the commands and Appendix H the attested pin.

Finally, an automated writing engine revised this manuscript and an image generator produced Figure 1; the figure is qualitative, and all quantitative content remains artifact-derived. Build, prompt, semantic-audit, and authorship provenance are retained in the accompanying ledgers.

The resulting contribution is therefore a replayable operability audit with an explicit construction history and calibration gap, not a validated classifier of scientific quality.

References

James Cheney, Laura Chiticariu, and Wang-Chiew Tan. Provenance in databases: Why, how, and where. *Foundations and Trends in Databases*, 1(4):379–474, 2009. doi: 10.1561/1900000006.

C. J. Clopper and E. S. Pearson. The use of confidence or fiducial limits illustrated in the case of the binomial. *Biometrika*, 26(4):404–413, 1934. doi: 10.1093/biomet/26.4.404.

Eelco Dolstra, Merijn de Jonge, and Eelco Visser. Nix: A safe and policy-free system for software deployment. In *Proceedings of the 18th USENIX Conference on System Administration (LISA XVIII)*, pp. 79–92, 2004.

Jessica Zosa Forde and Michela Paganini. The scientific method in the science of machine learning, 2019. ICLR 2019 Debugging Machine Learning Models workshop.

Timnit Gebru, Jamie Morgenstern, Briana Vecchione, Jennifer Wortman Vaughan, Hanna Wallach, Hal Daumé III, and Kate Crawford. Datasheets for datasets. *Communications of the ACM*, 64(12): 86–92, 2021. doi: 10.1145/3458723.

Yue Jia and Mark Harman. An analysis and survey of the development of mutation testing. *IEEE Transactions on Software Engineering*, 37(5):649–678, 2011. doi: 10.1109/TSE.2010.62.

Daniel Kang, Deepti Raghavan, Peter Bailis, and Matei Zaharia. Model assertions for monitoring and improving ML models. In *Proceedings of Machine Learning and Systems 2 (MLSys 2020)*, 2020.

Ruslan Kuprieiev, skshetry, Peter Rowlands, Dmitry Petrov, Paweł Redzyński, Casper da Costa-Luis, et al. DVC: Data version control — Git for data & models. Zenodo, 2020. Software archived on Zenodo; the DOI is the concept DOI that always resolves to the latest release (first archived 2020; current release 3.67.1 as of 2026-03-31).

Chris Lamb and Stefano Zacchiroli. Reproducible builds: Increasing the integrity of software supply chains. *IEEE Software*, 39(2):62–70, 2022. doi: 10.1109/MS.2021.3073045.

Joelle Pineau, Philippe Vincent-Lamarre, Koustuv Sinha, Vincent Larivière, Alina Beygelzimer, Florence d'Alché Buc, Emily Fox, and Hugo Larochelle. Improving reproducibility in machine learning research (a report from the NeurIPS 2019 reproducibility program). *Journal of Machine Learning Research*, 22(164):1–20, 2021.

Neoklis Polyzotis, Martin Zinkevich, Sudip Roy, Eric Breck, and Steven Whang. Data validation for machine learning. In *Proceedings of Machine Learning and Systems 1 (MLSys 2019)*, 2019.

Sebastian Schelter, Joos-Hendrik Böse, Johannes Kirschnick, Thoralf Klein, and Stephan Seufert. Automatically tracking metadata and provenance of machine learning experiments, 2017. Workshop paper presented at the Machine Learning Systems Workshop at the 31st conference on Neural Information Processing Systems (NIPS 2017); the NeurIPS 2017 conference tag is confirmed on the Amazon Science publications page. No DOI or publisher page numbers exist; see citation lock addendum.

Walter A. Shewhart. *Economic Control of Quality of Manufactured Product*. D. Van Nostrand Company, New York, 1931.

Miles Turpin, Julian Michael, Ethan Perez, and Samuel R. Bowman. Language models don't always say what they think: Unfaithful explanations in chain-of-thought prompting. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023.

Manasi Vartak, Harihar Subramanyam, Wei-En Lee, Srinidhi Viswanathan, Saadiyah Husnoo, Samuel Madden, and Matei Zaharia. ModelDB: A system for machine learning model management. In *Proceedings of the Workshop on Human-In-the-Loop Data Analytics (HILDA'16)*, 2016. doi: 10.1145/2939502.2939516.

Matei Zaharia, Andrew Chen, Aaron Davidson, Ali Ghodsi, Sue Ann Hong, Andy Konwinski, Siddharth Murching, Tomas Nykodym, Paul Ogilvie, Mani Parkhe, Fen Xie, and Corey Zumar. Accelerating the machine learning lifecycle with MLflow. *IEEE Data Engineering Bulletin*, 41(4): 39–45, 2018.

A Reproduction commands

All sixteen command lines are read-only with respect to audited archives, require no GPU, and run on CPU in seconds. Paths are relative to this workspace; neighbour-seat inputs use role labels. Fourteen lines are certificates, one composes them, and one re-hashes every sha16 literal and rejects residual publication placeholders. The five field-audit commands follow; Table 5 lists the remaining R41 certificates, and the R52 additions appear before the single entry point. Independent-process records cover the R42 eleven- certificate edition and the R52 current chain; neither is party-independent.

Six-case ledger certificate (Panel A).

```
python3 analysis/audit_gate.py --a4-verify \
  --f5-yaml 'OWNER_SEAT_G_WORKSPACE/experiments/'\
     'qgate_tailturnover/qs/p7_qgate_dec_r1_fb2_v2.yaml' \
  --f5-spec-name p7-qgate-dec-r1-fb2-v2-r120-mgr \
  --f5-expected __r29_pin__FINAL_FIRE_LANDED
```

Expected: one `[OK]` line per ledger case plus one for the lineage arm, and exit code 0. A `MISMATCH` line means either that the audited archive changed or that the ledger is stale; by PRIM3 the second case is itself a self-FAIL of this ledger.

Table 4: Certificate reading against the cell's 22 record-time pins, preserved as the R41 historical snapshot: at R41 all $7/7$ unit verdicts recomputed unchanged. The current divergence is stated immediately below; `code/verify_certificate.py` attests only the canonical pin of Appendix H. Counts: `analysis/REPLAY_LOG_R41.json`.

Reading	Count
Byte-stable	20
Drift with registered cause	2

Decisive cell (Panel B).

```
./run_audit_gate_d1.sh
```

Expected: `group_verdict=PASS` with the per-unit verdicts of Panel B. The runner only reads the audited archives and only writes inside its own results directory. One point of this is load-bearing for Section 2.6 and we state it exactly: a re-run re-pins every audited artifact at its *current* hash, but it writes those fresh pins into the sidecar `audit_gate_d1_report_current.json` and never into the frozen report, whose bytes the recorded verdicts are anchored to. So the two files are meant to differ once an audited archive changes, and comparing them is a measurement of the audited archives rather than of the runner (Section 3.4). The frozen report is read-only history; if a re-run ever alters it, that is a defect in the runner, and one such defect occurred and was rolled back in this round (Table 8).

Verdict-state projection (Table 1; registered map).

```
python3 code/project_verdict_states.py
```

Expected: `audit_gate_d1_report_projected_r41.json` beside the frozen report, every unit carrying a `state_verdict`, an empty `unmapped` list, and exit code 0. The script reads the frozen report and the registered map and writes only the projected sidecar. A non-empty `unmapped` list, and the resulting non-zero exit, is the PRIM5 enforcement mechanism of Section 2.7.

Synthetic fixtures (Table 11).

```
python3 code/audit_gate_fixtures.py
```

Expected: eight `[OK]` lines, `all_match:true` in `analysis/FIXTURE_RESULTS_R41.json`, and exit code 0. The fixtures are the only archives in this note that we authored ourselves; the runner reads no external archive.

Write-time replay matrix (Figure 3).

```
python3 code/make_replay_matrix.py
```

Expected: `analysis/REPLAY_MATRIX_R33.json` with the per-unit pin classification, and the figure source PDF. This script re-hashes pinned artifacts and re-runs the six-case certificate; it never writes into an audited archive.

Write-time interpretation and record-time verification. Byte identity belongs to frozen inputs, whereas the audited seats continued working. A write-time re-pin therefore locates and hashes today's files without re-adjudicating the recorded verdict. All ledger arms reproduce. At the R41 snapshot, all decisive-cell unit verdicts also reproduced; the latest replay instead changes the external live-manuscript unit while retaining the group PASS. The certificate check compares current bytes against record-time pins without replacing them, using `code/verify_certificate.py`.

Snapshot caveat. Table 4 is preserved byte-for-byte as the R41 record it reports. The R49 replay still returns the same anchor tally and both negative controls, but its verdict-level check is no longer all-unit stable: `E_R33_threearm` changes from frozen PASS to current FAIL after the external seat-E manuscript ceased to contain the registered three-arm predicate. The other frozen unit verdicts

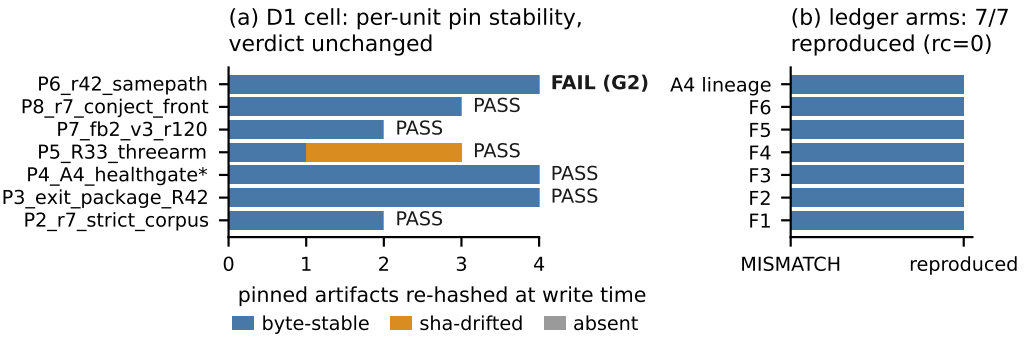

Figure 3: Write-time replay of both certificates, computed from on-disk artifacts by the script `make_replay_matrix.py` (Appendix A). **(a)** For each unit of the decisive cell, the artifacts pinned in the frozen report are re-hashed and classified as byte-stable, sha-drifted or absent; the verdict recorded in the frozen report is annotated at the right as a frozen-tier reading (not the current tier: seat E is currently FAIL), with the one G2 verdict shortened to `FAIL` (G2) — its exact string is the Panel B row of Table 1. **(b)** The six ledger cases plus the lineage arm, as reported by the re-run six-case certificate (exit code 0). Unit labels are the keys of the frozen report; the starred unit is the cell's positive control.

and the group PASS reproduce. The machine record is `analysis/CERT_VERIFY_R41.json`; this later divergence supersedes only the caption's present-tense reading, not its historical snapshot.

The drifted manuscript and PDF carry registered causes, so the certificate reads "anchors drift, explained" rather than byte-verified. A copied-anchor mutation without a cause emits `FAIL_ANCHOR_DRIFT_UNEXPLAINED`; adding a fabricated cause converts it to `PASS_ANCHOR_DRIFT_EXPLAINED`. Thus explained drift is only as trustworthy as its cause registry, and a reviewer may ignore that registry and read both drifted pins as unexplained FAILs. A second implementation agrees with the shipped checks, but remains an internal consistency check because one author wrote both.

The remaining certificates. Table 5 lists them with the artifact each writes and the property a reviewer should check in it. Each is one line, each reads only files inside this workspace, and each exits non-zero when its own registered expectation fails.

The R52 certificates.

```
python3 code/audit_gate_g1_extractor.py \
python3 code/g1_extractor_ablation.py \
python3 code/d2_decision_reachability.py
```

Expected: the extraction record of Table 17, the eleven-case ablation log with `pass=true`, and all nine calibration-design proof obligations satisfied.

All fourteen certificates at once.

```
python3 code/replay_all.py
```

Expected: fourteen `[ok]` lines and `all_commands_exit_zero:true` in `analysis/REPLAY_LOG_R41.json`, which also records the sha16 of every artifact the run produced. Two consecutive runs of this entry point reproduce all thirteen of those hashes, the two figure PDFs included; that is a determinism property of our own certificates and not of the archives they audit, whose drift is the subject of Section 3.4.

Table 5: The remaining commands, their artifacts, and what to check. The first six are certificates and are included in the single entry point below; the last is a manuscript-level check and is deliberately excluded from it, because its input is this `.tex` file rather than the audit. The certificate verifier attests the canonical pin of Appendix H; it does not certify that current archive verdicts equal the frozen tier.

Command (prefix `python3`)	Writes	What a reviewer checks
`code/make_prim3_fixtures.py`	the five `fixture_prim3_*` archives and their payloads	one archive per anchor state; regeneration is byte-identical
`code/verify_certificate.py`	`analysis/CERT_VERIFY_R41.json`	attests the canonical pin of Appendix H; the 22 record-time anchors tally as 20 byte-stable and 2 drift-with-cause, and both controls behave as registered
`code/gate_amendment_diff.py`	`analysis/GATE_AMENDMENT_DIFF_R41.json`	the two group-rule editions disagree in both directions over the enumerated verdict multisets, so neither is a strengthening of the other
`code/audit_gate_second_impl.py`	`analysis/DUAL_IMPL_CONSISTENCY_R41.json`	five checks re-implemented from the specification agree with the shipped ones, including the closed-form and bisected Clopper–Pearson limits
`code/make_coverage_matrix.py`	`analysis/COVERAGE_MATRIX_R41.json` and Figure 2	every cell is derived from a cited artifact; the zeros are the admissions of Section 3.3
`code/cp_design_numbers.py`	printed table	the bounds quoted in Sections 2.2 and 5, recomputed from the exact binomial tails
`code/check_paper_numbers.py`	`analysis/PAPER_NUMBER_SYNC_R41.json`	every 16-hex literal in this manuscript is either the current hash of the file it names or a declared record-time pin

B ARTIFACT ANCHORS

Tables 6 and 7 list the in-house artifacts that carry the claims of this note. They are run artifacts, not peer-reviewed literature, and are cited by path and sha16 only. Each hash is the leading 16 hexadecimal digits of the file's SHA-256, and — per Section 2.6 — each row is either a current *write-time re-pin* or is explicitly scoped to the earlier judge edition in its role/block. None silently substitutes for a record-time pin; the anchors whose two values disagree are listed separately in Table 8 with the cause of each difference. Every sha16 literal is checked by `code/check_paper_numbers.py`, which re-hashes current rows and rejects any other literal unless it is a declared record-time or edition pin, so a stale anchor in a note about anchors is a mechanical error rather than a proofreading one. One row is an exception by construction and we name it: the execution log embeds the decisive cell's wall clock, so re-running the certificates advances its hash even when all thirteen artifacts it records are byte-identical. Its pin is the value at the moment this version was compiled, and a mismatch there is the expected behaviour of a log rather than a stale anchor. Paths in every block but the last are relative to the auditing workspace; the last block is a neighbour seat's tree and carries its own prefix, because those files have never lived in our workspace and a workspace-relative reading of them would resolve to nothing.

Table 6: Write-time re-pinned anchors for the claims of Sections 2 and 3, part one. Record-time differences: Table 8. Reading: the role and block identify any edition-scoped pin rather than presenting it as current.

Artifact	Role	sha16
analysis/AUDIT_LEDGER_R27.json	six-case ledger base	64bf94507159fec4
analysis/AUDIT_LEDGER_R29_AMENDMENT.md	F5 final-verdict pointer	9d9ee1916b44ebb1
analysis/audit_gate.py	six-case certificate, with lineage arm	0ed8657a6ab7dbf9
analysis/ENDPOINT_OPERABILITY_R30.md	G1 and G2 clauses as filed	3ce4124595fd9a27
experiment_registry/PROP_A_AUDIT_GATE_METHOD.md	method v1.1, primitive layer	1f74929c0bc52e60
analysis/METHODS_NOTE_SKELETON_v1.md	registered clause text (Section 5)	c199b26332eb744a
experiments/prereg/PREREG_AUDIT_GATE_D1.md	decisive-cell pre-registration	7a2d5306b734478f
code/run_audit_gate_d1.py	decisive-cell runner, R41 write-time pin	27a5bab33dc432e7
run_audit_gate_d1.sh	recompute entrypoint	b9125ff1db0e980d
healthgate_adoption/CONSUMPTION_CONTRACT.md	custody contract for the neighbour gate	daf309eab10ff724
Refine-1 additions: PRIM5 projection, the synthetic fixtures, and the round-2 certificates		
analysis/VERDICT_STATE_MAP_R41.json	registered verdict-state map (Appendix C)	e892db517faebfba
code/project_verdict_states.py	projection script, PRIM5	7813d2f45d28d4c2
code/group_state_rule.py	group readout, rule editions v2, v2b and v3	1f742054a965ea07
code/prim3_anchor_states.py	PRIM3 five-state classifier and truth table	5f2ed3aadfbf547e
code/make_prim3_fixtures.py	generator for PTSD-4 to PTSD-8	42b781bb118737c9
code/audit_gate_fixtures.py	fixture runner, recomputes each verdict	b0ee24a1aa7f60c7
analysis/FIXTURE_RESULTS_R41.json	fixture verdicts, 8/8 match	963a9795cfa42867
code/verify_certificate.py	certificate verification, two negative controls	51845d9ee0ba5dba
analysis/CERT_VERIFY_R41.json	R41 anchor-state tally snapshot	a5215080b9d9a2c2
analysis/DRIFT_CAUSES_R41.json	operator-supplied drift causes, with checks	f282afd8fbf27d5d
code/gate_amendment_diff.py	enumerates where the two group rules differ	a13e9f8cbcc998e4
analysis/GATE_AMENDMENT_DIFF_R41.json	the multiset enumeration (Table 12)	919fa4d5481567a8
code/audit_gate_second_impl.py	second implementation of five checks	5dab6aab6d91ba56
analysis/DUAL_IMPL_CONSISTENCY_R41.json	agreement of the two implementations	2cf59c90dda2a2de
code/make_coverage_matrix.py	coverage matrix (Figure 2)	5c890b50e03a32df
analysis/COVERAGE_MATRIX_R41.json	per-check live and fixture fire counts	5f07c848f36eb68f
code/cp_design_numbers.py	every Clopper–Pearson bound quoted here	0dd7441d0f740d7d
code/replay_all.py	R41 certificate entrypoint edition	9c82b795711119f8
analysis/REPLAY_LOG_R41.json	execution log, all commands exit 0; advances on every run	f0c64feb4340cab6
analysis/REPLAY_MATRIX_R33.json	replay measurement (Figure 3)	c874929fec1f90b1
code/check_paper_numbers.py	re-hashes every sha16 literal of this note	7b7866a9d91960e2
analysis/THIRD_PARTY_REPLAY_R41.json	independent-process replay record (Section 6)	8645cb181b5df08a

Where the two pin semantics disagree. Table 8 is the construction-history disclosure that Section 2.6 promises. Six lines now carry a write-time hash different from the value recorded when the verdict was first taken; four are the R31–R41 lineage disclosed in earlier versions, and the last two exist because the R42 judge-edition repair (Table 12, row four) rewrote the sidecar and added its projection tier. These six rows concern our auditing files; the separate seat-E archive-verdict drift and seat-C/seat-H extraction POSTPONEs are recorded in Appendix H. We list the pin changes rather than silently reprinting today's numbers, because a hash table that quietly tracks the present is exactly what PRIM3 rejects.

Table 7: Write-time re-pinned anchors, continued: the external-review repair, registered future cell, fixture inputs, decisive-cell outputs and read-only neighbour assets. Reading: the blocks separate current repair artifacts from frozen or edition-scoped records.

Artifact	Role	sha16
R42 external-review repair round		
code/run_audit_gate_d1.py	R42 runner edition: G2 as predicate, amended disjunct	9286a81b3f4c9f7a
code/group_state_rule.py	R42 edition: v2b rule transcribed, stale "strictly stronger" docstring corrected	1f742054a965ea07

Continued on next page

Table 6 (continued)

Artifact	Role	sha16
code/project_verdict_states.py	R42 edition: two projection tiers	7813d2f45d28d4c2
code/check_paper_numbers.py	R42 edition: R42 pins and counts registered	7b7866a9d91960e2
analysis/THIRD_PARTY_REPLAY_R42.json	independent-process replay, all eleven certificates, current tier (Section 6)	92da51c618da254a
code/baseline_naive_audit.py	strawman baseline auditor (Section 3)	7529c77ea63be55b
analysis/NAIVE_BASELINE_R42.json	the baseline's flags and passes, ablation table source	64d39ef57b928a01
R52 mechanical-extraction tier		
code/audit_gate_g1_extractor.py	mechanical G1/G2 extractor	17a2ca85ed545335
code/audit_gate_g1_parsers.py	one parser per archive schema	dc7ebfb918acae30
code/g1_extractor_ablation.py	eleven delete/mutate and constant-grep checks	e3cc1a30ed52c26a
code/d2_decision_reachability.py	calibration decision-reachability proof	cee3b44f4a2e9663
analysis/D2_DECISION_REACHABILITY_R52.json	all nine proof obligations hold	3cbfe58aba315261
code/replay_all.py	fourteen-certificate entry point	1fa291b75d9c804f
analysis/THIRD_PARTY_REPLAY_R52.json	current-chain independent-process replay	62265a41e55a7928
Registered future cell under experiments/prereg/		
PREREG_AUDIT_GATE_D2_CALIBRATION.md	calibration design, registered and not run (Section 5)	2c56d7c3c330574b
Fixture inputs under experiments/fixtures_r41/		
fixture_g1_missing.json	G1 quintuple incomplete	a238d88d34dd127e
fixture_prim1_noncrossing.json	PRIM1 non-crossing envelope	65b52c177e9d5866
fixture_prim4_unreachable.json	PRIM4 unreachable primary leg	0832d5d9e5a4d533
primary_leg.csv	the zero-row primary leg itself	9b3b65548e77a25c
fixture_prim3_byte_stable.json	PRIM3 anchor state 1 of 5	b5c1ab090ccfcfe7
fixture_prim3_drift_explained.json	PRIM3 anchor state 2 of 5	26f7480b4b6cc3f4
fixture_prim3_drift_unexplained.json	PRIM3 anchor state 3 of 5	6caa7aff50f9c61f
fixture_prim3_absent_marked.json	PRIM3 anchor state 4 of 5	d247fac203e452da
fixture_prim3_absent_unmarked.json	PRIM3 anchor state 5 of 5	96772e17a43a8021
anchor_payload_stable.txt	the byte-stable payload itself	808c6aea0068823c
anchor_payload_drift_explained.txt	payload revised, cause recorded	615ad1d5820fe4ba
anchor_payload_drift_unexplained.txt	payload revised, no cause	194160d8d550aed4
Decisive-cell outputs under experiments/results/audit_gate_d1/		
audit_gate_d1_report.json	frozen cell report, read-only history	d91f180ea59edc39
audit_gate_d1_report_current.json	sidecar written by a re-run (R42 tier)	f47224d6a9957472
audit_gate_d1_report_projected_r41.json	the frozen report with projected states and the group readout (R31 tier, unchanged)	aba51fa7f7b3a8b9
audit_gate_d1_report_projected_r42.json	the current sidecar with projected states, group readout and the judge-version drift disclosure	675deaa246e2b9b6
audit_gate_d1_summary.md	cell summary, with the self-FAIL note	7b551cc6c416b372
Neighbour seat D's readout-health-gate assets (anonymized, frozen, read-only; internals not restated here)		
healthgate.py	gate implementation	9f4bc2711502f358
PREREG_A4.md	its pre-registration	d1192e1383f1cf90
healthgate_report.json	positive arm archive	9bf250c74cbfd2fd
healthgate_failarm.json	dead-parser fail arm	3d5339db6c4ce984

Table 8: Record-time pin versus write-time re-pin, a construction finding for the six anchors where they differ. Every other anchor in Tables 6 and 7 is byte-stable against its record-time value, including all four neighbour-seat assets and the six-case checker. "Verdict affected" asks whether the verdict the anchor carries changed, not whether the bytes changed.

Anchor	Record-time pin	Write-time re-pin	Recorded cause	Verdict affected
`PREREG_AUDIT_GATE_D1.md`	`cd80d58efaee05aa`	`7a2d5306b734478f`	the pre-registration's own text was amended after the first run to record the self-FAIL and its correction (Table 12). A third value also circulates and we name it rather than let a reviewer trip on it: the prereg's §7 prints an internal self-pin `3380a46becbd17bd`, recorded at the freeze moment before the summary's second back-fill of hashes, whose value the prereg declares authoritative for itself — so record-time, internal and write-time are three snapshots of one file across its own amendment	no
`code/run_audit_gate_d1.py`	`88922152856c9e98`	`9286a81b3f4c9f7a`	corrected in refine-1 to write its output to a sidecar instead of over the frozen report; repaired again in R42 (Table 12, row four): G2 became a predicate and the group second disjunct was amended, so this anchor is the one place where a write-time pin names a different *judge edition*, not only different bytes — the frozen report remains the registered record of the R31 judge	no (R31 verdicts recomputed against the R31-era sidecar; the R42 edition agrees on every unit)
`run_audit_gate_d1.sh`	`6446b522fa19d5c6`	`b9125ff1db0e980d`	extended to pin the sidecar and the projected report alongside the frozen one	no
`audit_gate_d1_report.json`	`4dcfcb41a3984e94`	`d91f180ea59edc39`	audited-archive pins inside the report advanced between the first run and the replay-matrix round; separately, a first attempt at the refine-1 projection wrote state fields into this file in place, which was detected and rolled back, and is why the projection now emits a separate file	no
`audit_gate_d1_report_current.json`	`92536d3a32426685`	`f47224d6a9957472`	the sidecar is a write-time object by design, so "record-time" here means the R41-era value printed in earlier versions: the R42 judge edition rewrote the sidecar (row four of Table 12); the frozen report above is untouched	no — the verdict *strings* reproduce, the bytes legitimately differ
`audit_gate_d1_report_projected_r42.json`	—	`675deaa246e2b9b6`	new in R42, so no record-time value exists to differ from: the R42-tier projection, carrying `recompute_diff_zero=false` and `judge_version_drift`; the R41-tier projection is unchanged at `aba51fa7f7b3a8b9`	n/a

C THE REGISTERED VERDICT-STATE MAP

Record-time anchors and write-time re-pins. Certificate semantics bind only to the locations and hashes pinned when a verdict was recorded. The sha16 values in Tables 6–7 are instead write-time re-pins that help a reader find today's files; they do not re-adjudicate the record. When the two differ, the difference describes later file drift, not an incorrect original verdict. Table 8 records every such difference and its cause. The runner writes a current sidecar and never mutates the frozen report.

Full freeze history. The decisive cell's first implementation treated a unit-level FAIL as a collapsed group state and emitted `FAIL_VACUOUS_STATES`, although the registered predicate already ac-

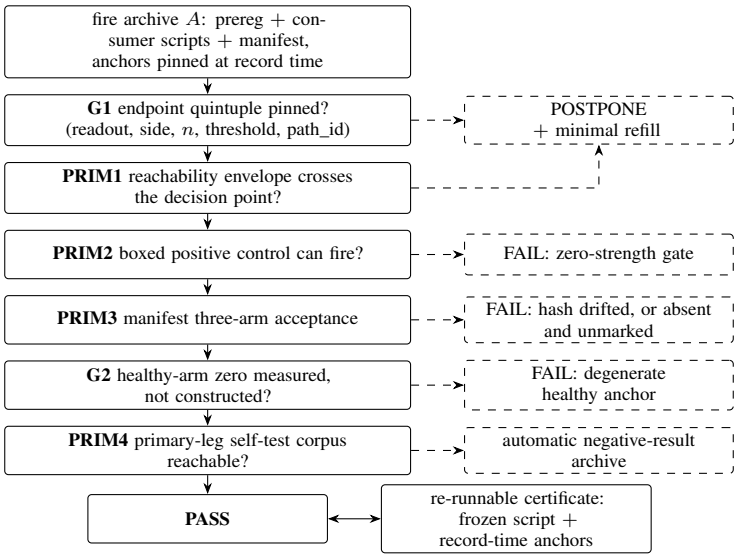

Figure 4: Schematic of the AUDIT-GATE decision path; the figure carries no quantitative content. Solid arrows are the operability chain, dashed arrows are terminating verdicts. Every terminal verdict, PASS included, is emitted together with a re-runnable certificate, and POSTPONE carries the minimal refill that would make the endpoint decidable. Symbols match Section 2; the observer-round parameter of Section 2.4 ($K = 2$ by default) applies to consumption verdicts rather than to the chain drawn here.

cepted that multiset. The repair moved the implementation toward the pre-registration, but happened after observation. Because the first edition was overwritten before pinning, its verdict survives only in the cell summary and cannot be replayed; this is the self-FAIL disclosed in Section 2.6. The map was later extended to strings that its first edition omitted, PRIM3 was made total over the anchor facts, and G2 was repaired from an assignment to a predicate. Table 12 gives the exact editions, locations, and re-executability.

Projection and legacy predicates. PRIM5 strips provenance markers before lookup, fails closed on an unregistered string, keeps unit and group codomains separate, and records how every string was exercised. For a lifecycle source p, `FAIL_TELESCOPE` is emitted when both a review-only transition branch and an entering-refine branch occur literally in p; the complement is `TELESCOPE_CLOSED`. The group readout is likewise separate: execution PASS requires mapped unit strings, more than one projected state, behaving controls, and reproducible execution, while the archive-operability tally remains visible beside it. Exhaustive comparison of the shipped and registered group rules finds disagreement in both directions; the observed corpus changes under neither (Appendix E).

Table 9: The registered verdict-state map (`analysis/VERDICT_STATE_MAP_R41.json`, sha16 `e892db517faebfba`): all 26 unit strings the checkers can emit, with the state tally 13 FAIL, 9 PASS and 4 POSTPONE. The 3 strings of the second codomain, which are verdicts about an audit run rather than about an archive, are Table 10. Tags: *live* = emitted on the audited corpus of Table 1; *fixture* = exercised only on a synthetic archive; *diagnostic* = printable but never archived as a case verdict; *historical* = superseded; *edition-gated* = emitted by an earlier edition of the runner only. Every POSTPONE row is tagged *fixture*, *diagnostic* or *historical*, which is the coverage gap of Section 3.3 in its most compact form.

Machine string	State	Case	Predicate, and how it is exercised
`PASS_OPERATIONAL`	PASS	F4	all registered computors present and non-empty *(live)*
`PASS_OPERATIONAL_EXTERNAL_CANDIDATE`	PASS	A4	four lineage pins match sha16, positive gate passes and is usable, fail arm all-FAIL and not usable *(live)*
`PASS`	PASS	D1 units	the per-unit acceptance criteria of the D1 runner *(live)*
`FINAL_FIRE_LANDED`	PASS	F5 amend.	amendment spec name pinned in the job file and verdict file present; emitted behind the `__r29_pin__` marker *(live)*
`REACHABLE`	PASS	F1	at least one computor hit *(diagnostic)*
`PATCHED_OK`	PASS	F2	the empty-set fallback has been patched *(diagnostic)*
`TELESCOPE_CLOSED`	PASS	F6	the telescope predicate is false, i.e. the repair landed *(fixture)*
`ANCHOR_BYTE_STABLE`	PASS	PTSD-4	PRIM3 anchor state 1 of 5: the recomputed digest equals the record-time pin *(live: 20 of 22 pins of Figure 3; also PTSD-4)*
`PASS_ANCHOR_DRIFT_EXPLAINED`	PASS	PTSD-5	PRIM3 anchor state 2 of 5: digest differs, a cause is registered for that path, and the unit verdict recomputes unchanged *(live: the 2 remaining pins; also PTSD-5)*
`FAIL_MECHANICALLY_UNREACHABLE`	FAIL	F1	the registered readout has zero computor hits in every consumer script *(live)*
`FAIL_EMPTY_SET_PASSED_GATE`	FAIL	F2	an empty-denominator fallback (zero total, rate falls back to 1.0) is present and unpatched *(live)*
`FAIL_ABSENT_UNMARKED`	FAIL	F2, F5	a referenced artifact is absent and carries no ABSENT marker *(live)*
`FAIL_ABSENT_UNMARKED_4_OF_5`	FAIL	F3	at least four of five recorded consumer paths absent at their recorded locations, with no ABSENT marker at record time *(live)*
`FAIL_TELESCOPE`	FAIL	F6	$T(p)$ of Section 2.7: both transition branches occur in one tool source *(live)*
`FAIL_DEGENERATE_HEALTHY_ANCHOR`	FAIL	seat F	the healthy arm's zero holds by construction and the threshold's only real anchor is a mismatched-path arm *(live)*
`FAIL_SPEC_DRIFT`	FAIL	F5 legacy	the legacy spec name is no longer pinned in the job file after an authorised rename *(live, informational: the ledger records this as a pointer to the amendment, not as an audited-archive FAIL, which is why the legacy invocation's exit code is not treated as a verdict)*
`FAIL_MISSING_COMPUTOR`	FAIL	F4	any registered computor missing or empty; the mirror of F4's pass *(fixture)*
`FAIL_LINEAGE_DRIFT`	FAIL	A4	a lineage pin's sha16 drifted, or positive/fail-arm content drifted *(fixture)*
`FAIL_FINAL_UNANCHORED`	FAIL	F5 amend.	the amendment spec name is not pinned, or the verdict file is absent *(fixture)*
`FAIL_NULL_PRIMARY_LEG`	FAIL	PTSD-3	PRIM4: the registered primary self-test leg has zero scorable rows, so the archiving verdict holds and relabelling the leg is forbidden *(fixture; zero live occurrences)*
`FAIL_ANCHOR_DRIFT_UNEXPLAINED`	FAIL	PTSD-6	PRIM3 anchor state 3 of 5: digest differs and no cause is registered for that path *(fixture; also the mutation control of Section 3.4)*
`FAIL_NO_ROPE_ARM`	FAIL	seat H	the registered rope arm is absent or under-n in the smoke report *(edition-gated: emitted by the runner edition of the first run, whose group-level misclassification is Table 12; the current frozen edition's seat-H unit passes)*
`POSTPONE`	POSTPONE	D1 unit; fixtures	a unit-level exception, or registration incompleteness; always carries the minimal refill *(fixture; zero occurrences on the live corpus)*
`STALE_NOT_REPRODUCED`	POSTPONE	F3	fewer than four stale paths absent, so either the archive changed or the ledger is stale — and by PRIM3 the second reading is a self-FAIL of this ledger *(diagnostic)*
`POSTPONE_ANCHOR_ABSENT_MARKED`	POSTPONE	PTSD-7	PRIM3 anchor state 4 of 5: the pinned path is absent and an ABSENT marker was written at record time; the refill is the whole file *(fixture)*
`TBD_PROVISIONAL`	POSTPONE	F5 ledger	an in-flight fire attempt pending an authorised landing; a ledger state rather than a checker output at this round *(historical, superseded by the r29 amendment)*

Table 9 is the whole of the map that PRIM5 (Section 2.7) requires, transcribed from the registered file. Reading it is the only way to check Table 1 without running the projection script, and it is also the place where the framework's own coverage is visible: the right-hand tags say, string by string,

whether the live corpus ever emitted it. Four conventions are worth stating. The bare string `PASS` is a verdict string as well as a state name, so the identity map on it is registered explicitly rather than assumed. The `__r29_pin__` prefix is stripped before lookup, so it needs no entries of its own. A string tagged *diagnostic* is one a checker can print but the ledger never archives as a case verdict; it is registered anyway, because PRIM5 closes the domain over what the checkers *can* emit, not over what they happened to emit. And the map has a second codomain, Table 10: those three strings are verdicts about an audit run, they are produced by a different rule from the unit strings, and the one time we conflated the two levels the result was the false kill of Table 12. Where a row names a fixture, the identifier is the one the runner emits, since that is the executed artifact.

Table 10: The second codomain of the same registered map: the 3 group strings (2 FAIL, 1 PASS), which are verdicts about an audit *run* rather than about an archive and are produced by a different rule from the unit strings of Table 9. Keeping the two codomains in separate blocks is not cosmetic: the one time we conflated the levels, the result was the false kill of Table 12.

Machine string	State	Case	Predicate, and how it is exercised
`EXECUTION_VALID`	PASS	D1 group	no unit string is unmapped and the projected multiset is not one repeated state, i.e. the audit ran and discriminated *(live; renamed from the legacy `AUDIT_EXECUTION_PASS` label without changing semantics)*
`FAIL_VACUOUS_STATES`	FAIL	D1 group	the projected unit multiset collapses to a single state, so the audit did not discriminate on this corpus *(synthetic all-identical multiset, plus one recorded historical instance: the first run under the first-edition rule, Table 12 — that instance is recorded, not recomputable, the pre-fix edition being absent from disk, Section 2.6)*
`FAIL_UNMAPPED_VERDICT`	FAIL	D1 group	some unit string has no entry here and no prefix rule applies; this is the enforcement mechanism of rule (ii) *(never fired on the frozen report)*

D SYNTHETIC FIXTURES AND THE PRE-REGISTRATION DIFF

Table 11 is the per-fixture record behind Section 3.3, and Table 12 is the pre-registration diff that Section 2.6 commits us to. The two belong together: the first shows the framework's unexercised clauses firing on inputs we built, and the second shows the one occasion on which the framework's own gate had to be changed after seeing an observation.

The authored fixtures cover the branches absent from the live corpus. One tests an incomplete G1 registration, one searches the non-crossing PRIM1 envelope for its refill, and one exercises an unreachable primary leg. The remaining fixtures enumerate PRIM3 as a total function of path presence, digest equality, a recorded cause, and an ABSENT marker. Each archive carries one injected defect and an expected verdict; the runner derives the observed verdict and exits non-zero on disagreement. These are positive controls for mechanical operability, not estimates of performance on natural archives.

Table 11: The eight synthetic defect-injected fixtures. Each injects exactly one defect and carries its expected verdict as a registered field, so the runner (`code/audit_gate_fixtures.py`) compares rather than asserts; it recomputes each verdict from first principles, including searching for $n^\star$ rather than looking it up. PTSD-1 and PTSD-2 POSTPONE under G1's missing-field rule. PTSD-4 to PTSD-8 are one archive per row of the five-state PRIM3 anchor enumeration, and the runner also checks that the enumeration is total. Expected and observed verdicts match on all eight rows, and the run uses no GPU. Report: `analysis/FIXTURE_RESULTS_R41.json`, sha16 `963a9795cfa42867`.

ID	Injected defect	Verdict (expected = observed)	Minimal refill
PTSD-1	G1 quintuple registers `readout`, `side`, n but omits `threshold` and `path_id`	`POSTPONE`	the two named missing fields

Continued on next page

Table 11 (continued)

ID	Injected defect	Verdict (expected = observed)	Minimal refill
PTSD-2	PRIM1 envelope does not cross: $k = 0$, $n = 40$, LO side, $\theta = 0.03$, so $U = 0.0881$	`POSTPONE`	82 observation rows, reaching $n^\star = 122$
PTSD-3	PRIM4 primary self-test leg has zero scorable rows and is relabelled as a sensitivity analysis	`FAIL_NULL_PRIMARY_LEG`	none: the archiving verdict holds automatically
PRIM3 record-time anchor enumeration, one fixture per state (present?, digest equal?, cause recorded?, ABSENT marker?):			
PTSD-4	state 1 of 5: pinned artifact present, digest equals the record-time pin	`ANCHOR_BYTE_STABLE`	none
PTSD-5	state 2 of 5: present, digest differs, a cause is recorded for that path	`PASS_ANCHOR_DRIFT_EXPLAINED`	none; the cause entry is the obligation
PTSD-6	state 3 of 5: present, digest differs, no cause recorded	`FAIL_ANCHOR_DRIFT_UNEXPLAINED`	record a checkable cause, or the state stands
PTSD-7	state 4 of 5: pinned path absent, ABSENT marker written at record time	`POSTPONE_ANCHOR_ABSENT_MARKED`	re-materialise the file at its recorded path (quantum: whole file)
PTSD-8	state 5 of 5: pinned path absent, no marker	`FAIL_ABSENT_UNMARKED`	none: the FAIL stands until the file or a record-time marker exists

Table 12: Post-observation amendments to this framework's own registered objects, disclosed as findings of the construction, not as defects (Section 2.6). Three are design-level (gate and definition editions); the fourth is the external-review repair disclosed in Section 2.3. On the gate row the three rule editions are separated explicitly, because an earlier version of this note labelled the first-run implementation's semantics as the registered one — it was not: the registered predicate never counted "any FAIL as collapse", and the amendment moved the shipped rule *toward* the pre-registration. "Re-executable" asks whether the semantics named in that row can still be run from bytes on this disk today; one row answers no, and that row is this note's one recorded, non-recomputable verdict (Section 2.6).

Object and trigger	Superseded semantics	Current semantics	Where filed	Re-exec.
Group-gate diversity criterion; trigger: the first run's legitimate `FAIL_NO_ROPE_ARM` on the seat-H unit was read by the first-run code as a group-level collapse	**v1, first-run code** (*not* the registered text): any unit FAIL is read as a collapsed verdict set, hence group `FAIL_VACUOUS_STATES`. **v3, registered** (prereg §3, *never changed*): the state set is neither all-PASS nor all-FAIL nor all-POSTPONE — v3 already accepted the first run's multiset, so v1 was inconsistent with the prereg and the amendments moved *toward* it	**v2** (R31): a G2 caveat is a separate bucket counted as a detection; an ordinary FAIL still counts toward collapse. **v2b** (R42, current): v2 plus *any* FAIL-family string counts as a distinct observed state in the second disjunct — the reading v3 had all along	"correction (R31)" of `audit_gate_d1_summary.md`; the prereg's post-run amendment note; v2/v2b/v3 executable in `code/group_state_rule.py`	**v1: no**; v2, v2b, v3: yes
The registered verdict-state map, v1 to v2	the map registered the strings the two panels had emitted; the PRIM4 and anchor-state strings had no entry, so the projection script would have exited non-zero on them	every string any checker of this note can emit is registered, each tagged with how it was exercised (live, fixture, diagnostic, historical, edition-gated)	the `v2_changes` block inside the map file itself	yes
PRIM3, from three arms to a five-state enumeration	"exactly one of three arms applies": present with matching hash (PASS), present with drifted hash and no recorded cause (FAIL), absent without a marker (FAIL)	the same decision as a total function of four record-time facts, with five reachable states; the two added are drift-with-a-recorded-cause (PASS) and absent-with-a-marker (POSTPONE). Two anchors of our own certificate fall in the gap; the certificate check prints the tally with and without the cause registry, and no unit verdict differs under that anchor-state toggle (Section 3.4)	`code/prim3_anchor_states.py` and the state-map file	yes

Continued on next page

Table 12 (continued)

Object and trigger	Superseded semantics	Current semantics	Where filed	Re-exec.
G2 and three Panel-B predicates, from constants to predicates (external review, R42)	the seat-F verdict was an unconditional assignment while its inputs were read from keys absent from the cell file (every read `None`, none entering a condition); the seat-C/seat-E/seat-G acceptance conjunctions were near-tautological (one `RESOLVED` token; a substring matching "harm"; two-word presence)	G2 is a predicate over the audited cell's own fields (`bit_exact` true or `max_max` $= 0$ on the healthy arm); seat C verifies a registered per-slot manifest; seat E requires the bound string plus an explicit three-arm mention; seat G requires the trial id and the pinned spec name in the job file. At the R42 repair point, unit verdicts were unchanged on all seven units	`code/run_audit_gate_d1.py` R42 edition and the `judge_version_drift` field of the R42 projection; the R31 frozen certificate is untouched	yes

E INTERNAL CHECKS ON OUR OWN CHECKERS

Three measurements in this appendix are checks of the auditing code rather than of any audited archive. They are grouped here because they share one limitation: we wrote both sides.

The two group-rule editions are incomparable. Section 2.7 states the group readout as a registered predicate, and the decisive cell shipped a different rule. Their complete disagreement counts and smallest witnesses are in Table 13; neither rule refines the other.

Table 13: Bidirectional group-rule disagreement over the registered verdict-multiset domain. Reading: neither edition contains the other.

Accepted by	Rejected by	Count	Smallest witness
Registered predicate	Shipped rule	75	six PASS units plus one `FAIL_NO_ROPE_ARM`
Shipped rule	Registered predicate	21	six `FAIL` units plus one `FAIL_DEGENERATE_HEALTHY_ANCHOR`

The first witness is the R31 misfire pattern minus its G2 caveat, showing that the correction of Section 2.6 rescued only multisets containing a G2 verdict and left the shipped rule inconsistent with the registered text elsewhere. The second is an all-FAIL multiset that the pre-registration excludes as a vacuum and the shipped rule passes. The multiset our corpus actually produced lies outside both disagreement sets, so no verdict reported in this note changes under either rule — which is a fact about our corpus and not a defence of the rule.

A second implementation of five checks. `code/audit_gate_second_impl.py` re-implements five checks from their written specifications by deliberately different means: the string projection by longest-literal matching over the map file rather than by dictionary lookup; the PRIM3 anchor states by an explicit 16-row lookup table rather than by nested conditions; the Clopper–Pearson zero-column limit by bisecting the exact binomial tail rather than by the closed form of Section 2.2; the fixture verdicts by one purpose-written reader per fixture; and the group predicate by a pairwise diversity test rather than by the shipped rule. The two implementations agree on every unit, every truth-table row, every fixture and the group readout. The only numerical difference is $|U_{\text{closed}}(40) - U_{\text{bisect}}(40)| = 4.4 \times 10^{-16}$, with both routes returning $n^\star = 122$ and a refill of 82 rows. This catches only errors that survive one implementation and not the other; it is not independence.

Design numbers of the registered calibration cell. The bounds that fix the design of Section 5's calibration cell are computed by `code/cp_design_numbers.py` rather than asserted; the aligned detection and false-alarm designs are Tables 14 and 15. The first pass is a pilot that reports intervals only, while the confirmatory pass requires 36 archives per family. The pre-registration freezes the defect-free stratum at 60 with a one-fire allowance and forbids growing it after fires are seen.

Table 14: Per-family detection design under two-sided 95% Clopper–Pearson intervals.

Quantity	Registered value
Lower bound at $10/10$	0.6915
Detection criterion	0.90
Bound at $n = 35$ (last value below the criterion)	0.89997
First n at which the bound crosses the criterion	36

Table 15: Pooled defect-free false-alarm design at $n = 60$. Reading: the registered upper bound 0.10 tolerates exactly one fire; tolerating two requires $n = 70$, and tolerating three requires $n = 85$.

Fires	Clopper–Pearson upper bound
0	0.0596
1	0.0894
2	0.1153

F Remaining delineation axes and inherited anti-patterns

Axes D4 and D5. *D4, output type*: the neighbour ships a gate script plus a written list of anti-patterns, that is, governance experience; this note ships a primitive layer, a variable-rate fan-in ledger whose FAIL channel is demonstrated, and the methods note itself. *D5, relation to the model*: the neighbour serves the readout layer of one inference stack and has no fine-tuning or checkpoint-phase concept, whereas AUDIT-GATE is object-agnostic, and the archives it audits in Table 1 include fine-tuning differentials, checkpoint-margin calibration, evaluation-distortion probes and pure CPU pipelines. Overlap self-assessment, stated plainly: zero at the claim layer, roughly a quarter at the level of method vocabulary, since "gate", "control" and "envelope" are statistical common property.

The six inherited anti-patterns. Reused, not reproved: (i) an empty set must fail a gate rather than pass it; (ii) a rate-only test is blind to direction; (iii) a run identifier must be a field, not a directory name; (iv) a truncated output is not a wrong answer; (v) short-sample behaviour does not certify long-generation behaviour; (vi) static inspection cannot see a runtime crash. PRIM2 and PRIM3 are the mechanical forms of (i) and (iii); (iv) and (v) survive as reasons why a readout-layer gate remains necessary alongside this one.

Two clauses added from the in-run review pipeline. The list continues with two clauses that reached us as review output on other seats' gates rather than as our own findings; they are stated here as normative clauses of the audit, and their falsification conditions are in Section 5.

(vii) A structural reachability precheck is a *sufficient-rejection, necessary-but-not-sufficient-admission* gate: a pre-registered upper bound computed before ordering — for instance $\max(b + c)$ for a McNemar-framed test, or a permutation design's p_{floor} together with a tolerance table — proves only that the test is not mechanically unanswerable. It does not prove that admission was correct. A positive control that purports to demonstrate no false kill must itself survive clause (viii); otherwise its pass records a false admission.

(viii) Any reference arm or control fixture whose *load-bearing numeric fields* are byte-identical to another arm's, beyond the metadata divergence one would expect, FAILs the audit. The converse trap is the reason the clause is written on fields rather than on files: two arms with distinct bytes and distinct inodes can still carry the same measurement — in the case that prompted this clause, a run's `always` and `never` reference pair agreed byte-for-byte in 20 of 22 top-level fields — so the audit must compare numeric payloads field by field, and its object must extend past the outputs directory to every artifact consumed downstream, including probe and perturbation copies, pre-gate analyser outputs, and any record that references those cells. Byte-level and inode-level checks are blind to exactly this failure.

Provenance of clauses (vii) and (viii). Neither clause is ours, and both were revised at least once before being admitted here, so we record the chain rather than the conclusion alone. Clause (vii) descends from a reachability precheck note whose first version over-claimed the mechanism and was corrected by a teacher review; clause (viii) descends from a fault-reconciliation record on another seat, which established that the defect's real channel was a wiring equivalence between two reference arms rather than a directory copy, and that hash- and inode-level auditing is blind to it. A separate seat's own v2 correction retired a quantitative prior we had earlier attached to clause (vii) — an "about a quarter of cells are unreachable" figure that its authors withdrew — and we removed the number rather than restating it. The provenance records that carry this chain are organised in Table 16; the baseline and its addendum are two editions of the local clause record.

Table 16: Read-only provenance chain for clauses (vii) and (viii). The two sha16 values pin the local draft editions; the neighbour records are cited at their run-tree locations.

Record	Role
Fault-reconciliation memo of neighbour seat E (anonymized; retained in the run tree)	fault reconciliation behind clause (viii)
Corrected reachability analysis of neighbour seat C (anonymized; retained in the run tree)	corrected reachability analysis
Registered source design of neighbour seat C (anonymized; retained in the run tree)	registered source design
Local baseline clause draft (sha16 `2ed57f6b51b6ce2c`)	baseline clause draft
Local superseding addendum (sha16 `5ca2e2d3ade43a7d`)	superseding addendum

We hold none of the first three; they are cited as located, read-only neighbour records, and the numbers we dropped on their authority are dropped rather than re-derived.

G PER-UNIT ENDPOINT REGISTRATION IN THE DECISIVE CELL

The complete G1 rule is $e = (\text{readout}, \text{side}, n, \text{threshold}, \text{path\_id})$. A missing field makes the endpoint unregistered and returns POSTPONE with the smallest missing-field or observation-row refill that restores decidability. The path identity fingerprints the decoding or serving route, weight/configuration state, and readout parser or grader; a threshold is valid only within that identity. These last two fields were added after adjudication showed that sample count and threshold alone underdetermine the observation path.

Two tiers must be distinguished. In the frozen R31–R42 tier, the G1 quintuples and G2 answers are auditor transcriptions, not mechanical extractions: per-unit constants written after reading each archive. The frozen "$5/5$" therefore counts transcribed fields, not fields proved present. The current tier instead uses one parser per archive schema (`code/audit_gate_g1_extractor.py` and `code/audit_gate_g1_parsers.py`); unparseable or contradicted fields degrade to POSTPONE under G1's missing-field rule. Five units extract fully; seat C POSTPONEs because G2 is not mechanically parseable, and seat H POSTPONEs because its threshold is not. Eleven delete/mutate ablations plus constant-grep and identity controls pass in `experiments/results/audit_gate_d1/g1_extractor_ablation_log.json`. The runner consumes these evidence records directly, making the current sidecar the mechanical-extraction tier while the frozen report remains the transcription tier.

H CANONICAL PIN

This appendix is the single canonical statement of the audit claim and supersedes every historical tier for current-verdict reporting. The certificate verifier attests this pin: the canonical checker, its recorded input pins, the current mechanical-extraction verdict, the frozen verdict retained as history, and the replay interface. The decisive-cell current tally is 3 PASS, 2 FAIL, 2 POSTPONE; the frozen tally is 6 PASS, 1 FAIL, 0 POSTPONE. Table 18 is therefore single-valued with Table 17 and the tier caveats in Table 1.

Table 17: Two provenance tiers of the decisive cell. Panel A is the current mechanical- extraction tier, with current-verdict tally 3 PASS, 2 FAIL, 2 POSTPONE; Panel B is the frozen auditor-transcription record, tally 6 PASS, 1 FAIL, 0 POSTPONE. Frozen G1/G2 fields are auditor transcriptions, not mechanical extractions.

Panel A: current mechanical-extraction tier

Unit	Current verdict	G1 mech.	G2 mech.	Pins
D_A4_healthgate*	PASS	5/5	yes	4
C_exit_package_R42	POSTPONE (G2)	5/5	not parseable	4
E_R33_threearm	FAIL	5/5	yes	3
F_r42_samepath	FAIL_DEGENERATE_HEALTHY_ANCHOR	5/5	yes	4
G_fb2_v3_r120	PASS	5/5	yes	2
H_r7_conject_front	POSTPONE (threshold)	4/5	yes	3
A_r7_strict_corpus	PASS	5/5	yes	2

Panel B: frozen R31–R42 auditor-transcription record

Unit	Frozen verdict	G1 transcribed	G2 answered	Pins
D_A4_healthgate*	PASS	5/5	yes	4
C_exit_package_R42	PASS	5/5	yes	4
E_R33_threearm	PASS	5/5	yes	3
F_r42_samepath	FAIL_DEGENERATE_HEALTHY_ANCHOR	5/5	mech. (current)	4
G_fb2_v3_r120	PASS	5/5	yes	2
H_r7_conject_front	PASS	5/5	yes	3
A_r7_strict_corpus	PASS	5/5	yes	2

Table 18: Canonical pin: one row per decisive-cell unit and one block for the six-case ledger. The checker and input descriptions identify the recorded evidence; current and frozen verdicts are kept in separate columns.

Unit	Checker sha16	Input pins	Current verdict	Frozen verdict
Decisive cell; replay: `run_audit_gate_d1.sh`				
D_A4_healthgate*	9286a81b3f4c9f7a	owner-gate lineage	PASS	PASS
C_exit_package_R42	9286a81b3f4c9f7a	per-slot manifest	POSTPONE (G2)	PASS
E_R33_threearm	9286a81b3f4c9f7a	live-archive cell records	FAIL	PASS
F_r42_samepath	9286a81b3f4c9f7a	same-run cell records	FAIL (G2)	FAIL (G2)
G_fb2_v3_r120	9286a81b3f4c9f7a	owner-seat job pins	PASS	PASS
H_r7_conject_front	9286a81b3f4c9f7a	smoke-report records	POSTPONE (threshold)	PASS
A_r7_strict_corpus	9286a81b3f4c9f7a	corpus summaries	PASS	PASS
Six-case ledger; replay: `python3analysis/audit_gate.py`				
F1–F6 and A4	0ed8657a6ab7dbf9	Table 6 pins	matches frozen ledger	matches frozen ledger

Three scope clauses bind the pin. PASS certifies mechanical operability, not scientific correctness. POSTPONE follows only from the audit's missing-field rule: seat C lacks a mechanically established G2 answer and seat H lacks a certified threshold. Historical transcriptions and replay snapshots remain in the ledger as the archaeological record, not as competing current verdicts.

I FIGURE PROVENANCE

The four figures have three different provenances and we separate them, because a reader is entitled to know which pixels came from data and which from a generator.

Figure 4 is a programmatic schematic drawn from the section it illustrates and contains no quantitative content. Figures 3 and 2 are generated from on-disk artifacts by `code/make_replay_matrix.py` and `code/make_coverage_matrix.py`; no value in either is entered by hand, and re-running the scripts reproduces both, byte for byte — each script suppresses the embedded creation timestamp so that the figure PDFs are as reproducible as the JSON artifacts they are drawn from.

Figure 1 is a raster illustration produced with an image generator, and it is the only generated asset in the manuscript. Its status is qualitative: it carries no quantitative value, no measurement and

no experimental outcome, and nothing in the paper's argument rests on it. Three candidates were generated. The first was rejected on scientific-semantic grounds — it omitted the G2 check from the audit chain, which matters because G2 is the check that fires on the one detection in Panel B, and it showed the input archive with three components instead of four. The second passed the element audit and was the prior adopted version. The third, adopted here, preserves that version's semantics while making the PRIM5 projection an explicit box between the raw machine string and the three mutually exclusive states. Audited element by element against Section 2, it shows the four archive objects, the six checks in the order of Figure 4, PRIM5 outside and after that stack, the three state badges as its only outcomes, and a separate certificate path to a reviewer; no GPU appears, and the illustration carries no measurement or experimental outcome. The prompts, tool, candidates, semantic audits and adoption decisions are recorded in `build/image_generation_record.json`. Two further facts belong there rather than in a caption. An earlier round's attempt at this figure failed with a hard generator quota error, which is logged in the same file; the quota had been restored by the time this version was drafted. And the prompt asked for a specific image backend, but the tool exposed no backend selector, so we record what was requested and do not claim to know which model rendered the adopted asset.